Architectural Justification

Problem Statement #40 — Student Startup Crowdfunding Platform

Name : Pulla Jagadeeshwar Reddy SRN : PES1UG24CS347

Architecture selection : Microservices architecture

We're breaking the platform down into separate, independently deployable services — a Campaign & Pledge Manager, a Payment Service, a Milestone Review Service, and a Notification Service — all tied together through a shared UI and backed by a Campaign Database. This is shown in the component diagram.

We didn't go with a **Layered** architecture because it would tie everything together too tightly. If the layers are coupled, the whole app has to be redeployed and scaled as one unit, but pledging, milestone review, and payment settlement don't behave the same way at all — they get hit with different amounts of traffic and change at different rates, so bundling them together doesn't make sense.

We also ruled out **Client-Server**. Having one centralized server handle everything creates a single point of failure, and it doesn't scale well either. That's a real risk for a platform that needs to handle sudden spikes in pledges while still making sure milestone fund releases happen on time and stay auditable.

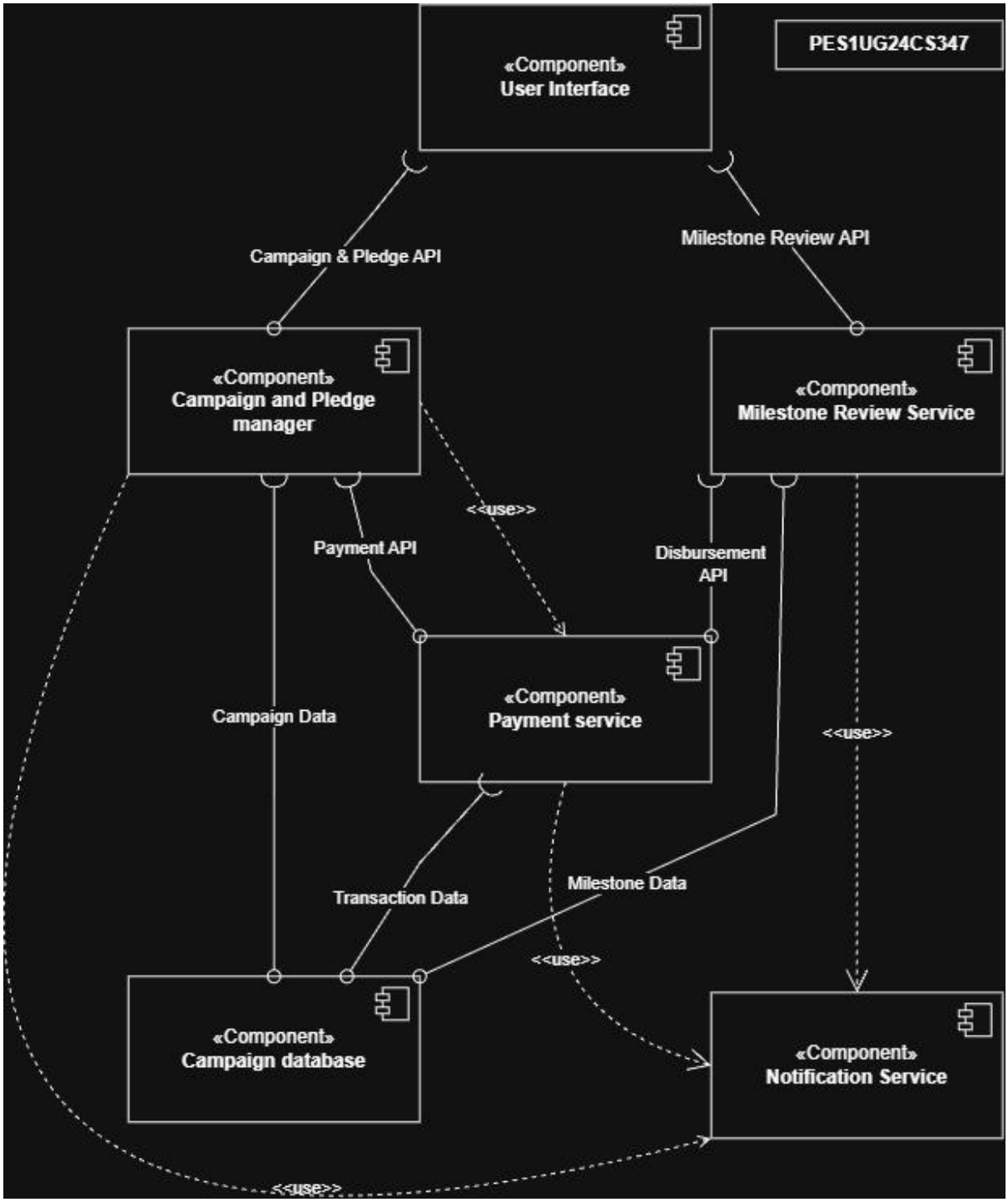
Reasons I picked Microservices:

1. Campaign browsing and pledging get a big traffic spike whenever a campaign launches — NFR-002 asks for support up to 2,000 concurrent users with response times under 2 seconds. Milestone review and payment activity, on the other hand, stay fairly low and steady. With microservices, we can scale just the Campaign & Pledge Manager during a launch instead of scaling the whole system, since the Payment Service and Milestone Review Service aren't under that same load.

2. The Milestone Review Service (where the faculty committee approves things) and the Payment Service (which actually sends out the money) are the parts of the system that matter most for auditing. Keeping them as separate services means we can update the campaign pages or UI as often as we want without having to touch, or risk breaking, the part of the system that deals with money.

3. Making Payment Service its own microservice also helps with PCI-DSS compliance (NFR-001). Instead of the entire app being in scope for handling card data, it's just this one locked-down service that deals with tokenized payments — so there's a smaller attack surface and less to audit.

4. When a campaign launches, we can just spin up more instances of the Campaign & Pledge Manager to handle the load, without touching the Payment Service or Milestone Review Service. This keeps things responsive during peak traffic (per NFR-002) without wasting resources scaling parts of the system that don't need it.


PES1UG24CS347
«Component»
User Interface
Campaign & Pledge API
Milestone Review API
«Component»
Campaign and Pledge manager
«Component»
Milestone Review Service
<<use>>
Payment API
Disbursement API
«Component»
Payment service
Campaign Data
<<use>>
Transaction Data
Milestone Data
<<use>>
«Component»
Campaign database
«Component»
Notification Service
<<use>>